Practical 5: K-Nearest Neighbors (KNN) Classification

Name: Aditya Bhende | **Class:** C | **Roll No:** 28 | **Batch:** C2

1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification

import warnings
warnings.filterwarnings('ignore')
```

2. Load Dataset

Run this cell and **upload** `diabetes.csv` when asked. If you skip the upload, the dataset is downloaded from a public URL instead.

```
df = pd.read_excel('diabetes (6).xlsx')
```

3. Exploratory Data Analysis (EDA)

```
df.info()
print('\nDuplicate rows:', df.duplicated().sum())
print('Null values:\n', df.isnull().sum())
df.describe().T
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB

Duplicate rows: 0
Null values:
 Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

	count	mean	std	min	25%	
Pregnancies	768.0	3.845052	3.369578	0.000	1.00000	3.00
Glucose	768.0	120.894531	31.972618	0.000	99.00000	117.00
BloodPressure	768.0	69.105469	19.355807	0.000	62.00000	72.00
SkinThickness	768.0	20.536458	15.952218	0.000	0.00000	23.00
Insulin	768.0	79.799479	115.244002	0.000	0.00000	30.50
BMI	768.0	31.992578	7.884160	0.000	27.30000	32.00
DiabetesPedigreeFunction	768.0	0.471876	0.331329	0.078	0.24375	0.37
Age	768.0	33.240885	11.760232	21.000	24.00000	29.00
Outcome	768.0	0.348958	0.476951	0.000	0.00000	0.00

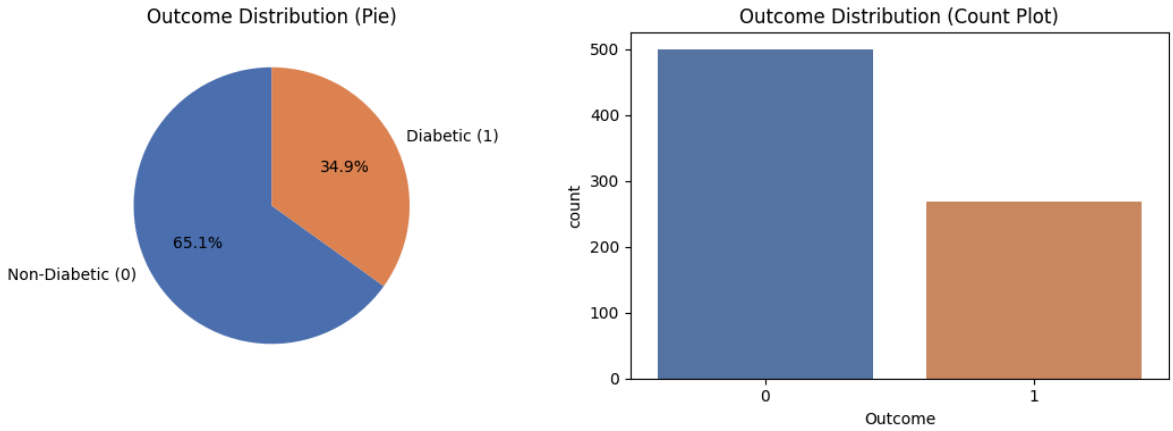
Output 1: Target Outcome Class Distribution

```
counts = df['Outcome'].value_counts()
print(counts)
print((counts / len(df) * 100).round(1))

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].pie(counts, labels=['Non-Diabetic (0)', 'Diabetic (1)'], autopct='%1.1
```

```
          colors=['#4C72B0', '#DD8452'], startangle=90)
ax[0].set_title('Outcome Distribution (Pie)')
sns.countplot(x='Outcome', data=df, ax=ax[1], palette=['#4C72B0', '#DD8452']
ax[1].set_title('Outcome Distribution (Count Plot)')
plt.tight_layout()
plt.show()
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
Outcome
0    65.1
Name: count, dtype: float64
```



4. Data Preprocessing: Invalid Zero Values

Glucose, BloodPressure, SkinThickness, Insulin and BMI cannot be 0 physiologically, so zeros here are missing measurements.

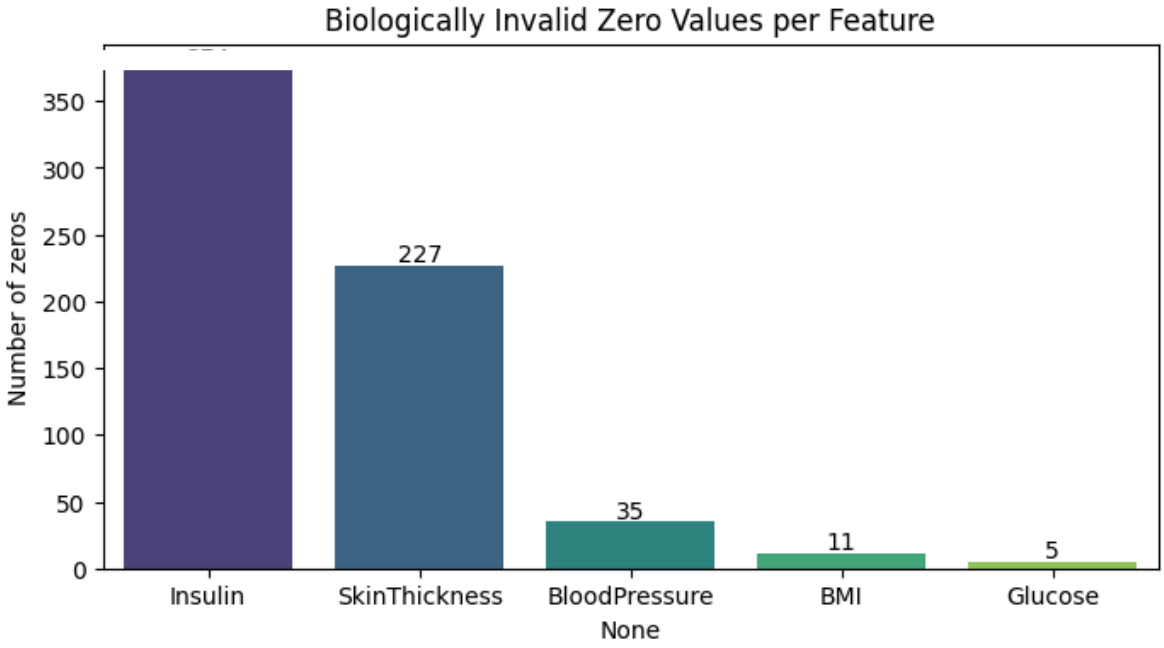
Output 2: Invalid Zero Value Identification

```
zero_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
zero_counts = (df[zero_cols] == 0).sum().sort_values(ascending=False)
print(zero_counts)

plt.figure(figsize=(8, 4))
ax = sns.barplot(x=zero_counts.index, y=zero_counts.values, palette='viridis
for i, v in enumerate(zero_counts.values):
    ax.text(i, v + 3, str(v), ha='center')
```

```
plt.title('Biologically Invalid Zero Values per Feature')
plt.ylabel('Number of zeros')
plt.show()
```

```
Insulin          374
SkinThickness    227
BloodPressure     35
BMI               11
Glucose            5
dtype: int64
```




```
df_clean = df.copy()
df_clean[zero_cols] = df_clean[zero_cols].replace(0, np.nan)
print('Missing values after replacing zeros:\n', df_clean.isnull().sum())

for col in zero_cols:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())

print('\nMissing values after median imputation:', df_clean.isnull().sum().s
df_clean.describe().T
```

```
Missing values after replacing zeros:
 Pregnancies                   0
Glucose                        5
BloodPressure                 35
SkinThickness                227
Insulin                      374
BMI                           11
DiabetesPedigreeFunction       0
Age                            0
Outcome                        0
dtype: int64

Missing values after median imputation: 0
```

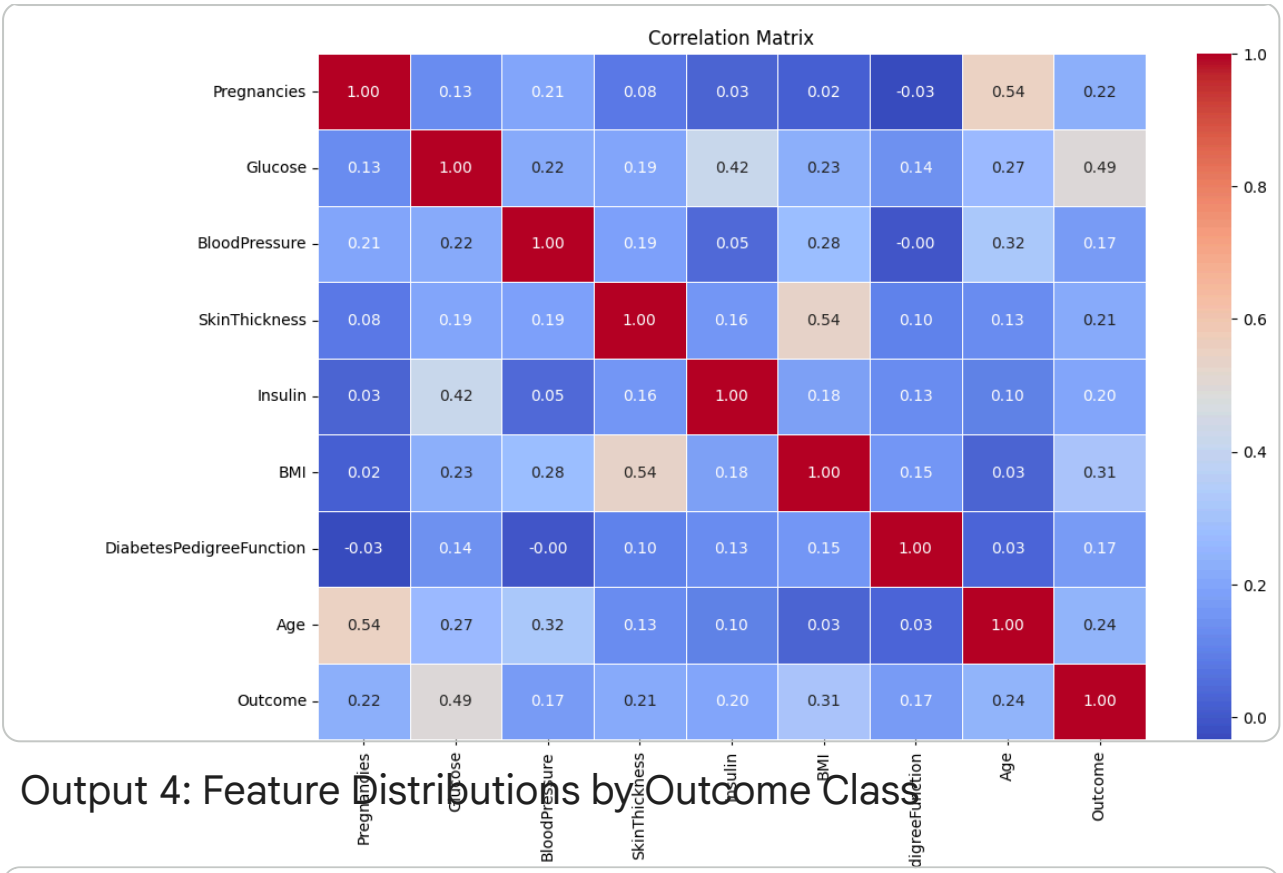
	count	mean	std	min	25%	
Pregnancies	768.0	3.845052	3.369578	0.000	1.00000	3.0
Glucose	768.0	121.656250	30.438286	44.000	99.75000	117.0
BloodPressure	768.0	72.386719	12.096642	24.000	64.00000	72.0
SkinThickness	768.0	29.108073	8.791221	7.000	25.00000	29.0
Insulin	768.0	140.671875	86.383060	14.000	121.50000	125.0
BMI	768.0	32.455208	6.875177	18.200	27.50000	32.3
DiabetesPedigreeFunction	768.0	0.471876	0.331329	0.078	0.24375	0.3
Age	768.0	33.240885	11.760232	21.000	24.00000	29.0
Outcome	768.0	0.348958	0.476951	0.000	0.00000	0.0

Output 3: Feature Correlation Heatmap & Target Relationships

```
plt.figure(figsize=(12, 8))
correlation = df_clean.corr()
sns.heatmap(correlation, annot=True, fmt='.2f', cmap='coolwarm', linewidths=
plt.title('Correlation Matrix')
plt.show()

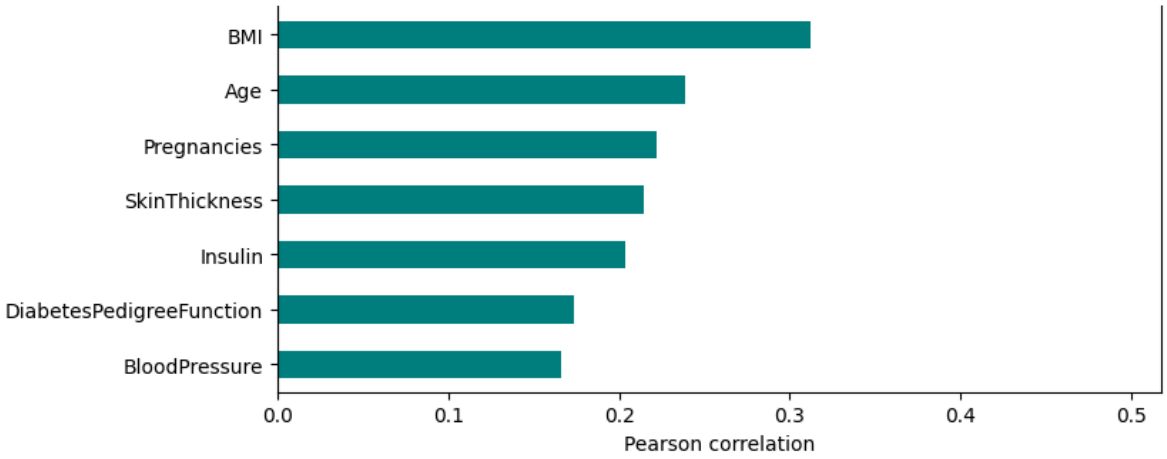
plt.figure(figsize=(8, 4))
correlation['Outcome'].drop('Outcome').sort_values().plot(kind='barh', color
plt.title('Correlation of Features with Outcome')
plt.xlabel('Pearson correlation')
plt.show()

print(correlation['Outcome'].drop('Outcome').sort_values(ascending=False).ro
```

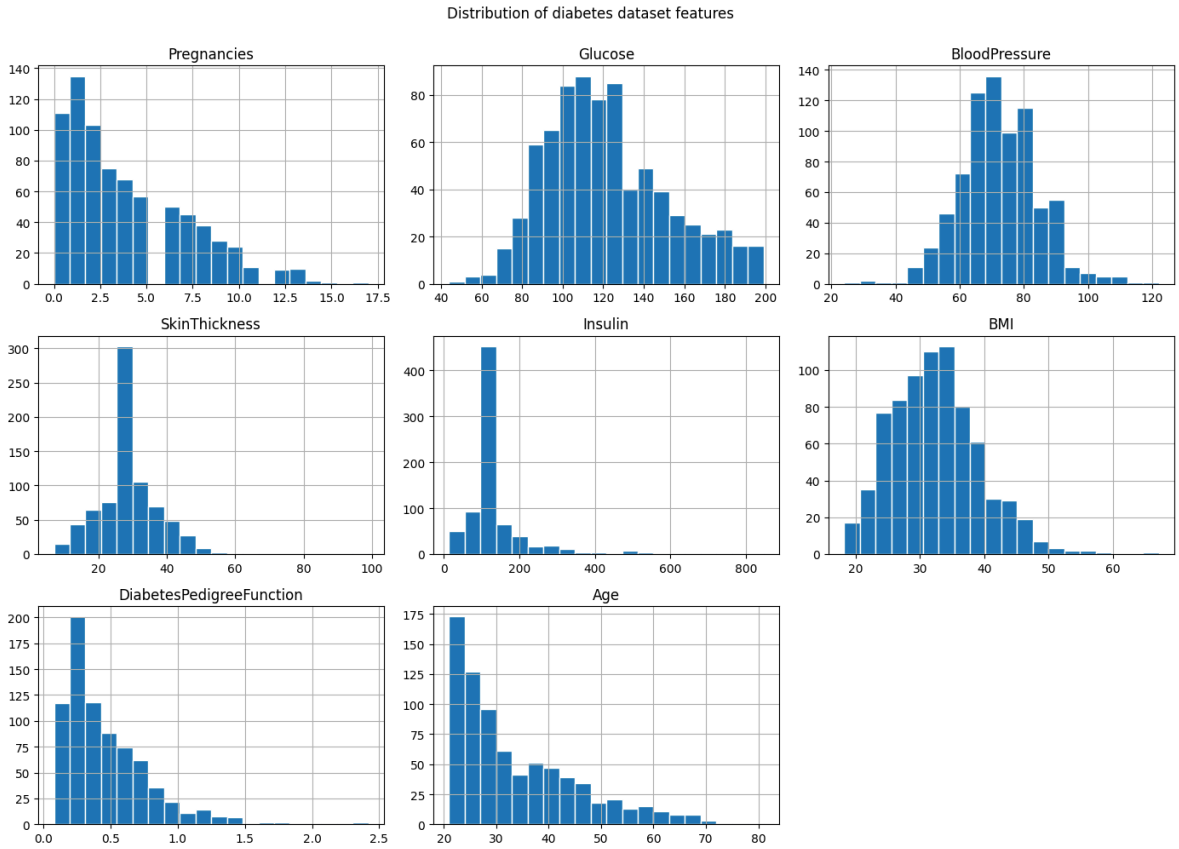


Output 4: Feature Distributions by Outcome Class

```
df_clean.drop(columns='Outcome').hist(bins=20, figsize=(14, 10), edgecolor='
plt.suptitle('Distribution of diabetes dataset features', y=1.0)
plt.tight_layout()
plt.show()
```

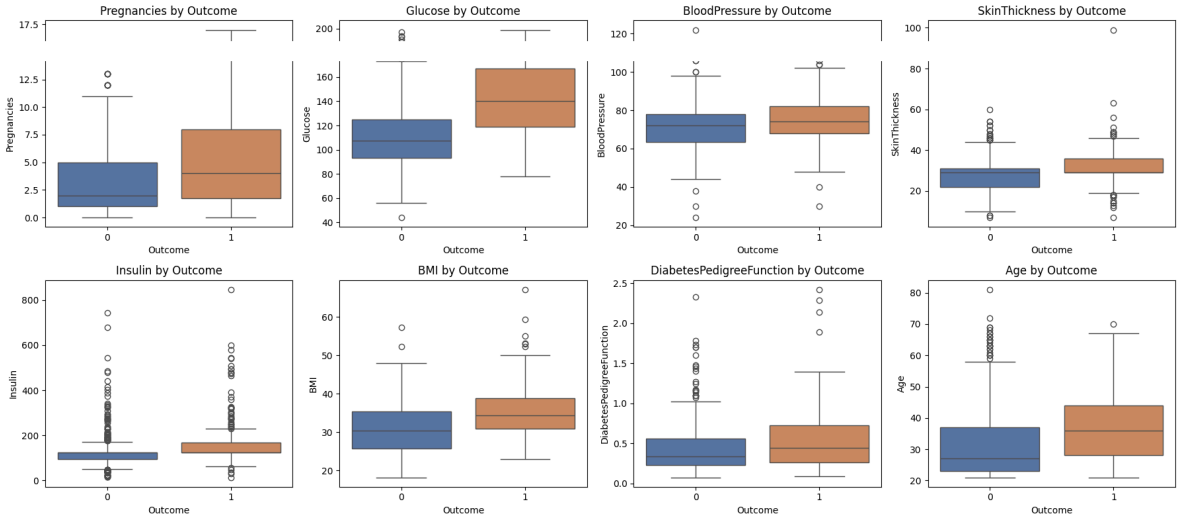



```
Glucose                     0.49
BMI                         0.31
Age                         0.24
Pregnancies                 0.22
SkinThickness               0.21
Insulin                     0.20
DiabetesPedigreeFunction    0.17
BloodPressure               0.17
Name: Outcome, dtype: float64
```

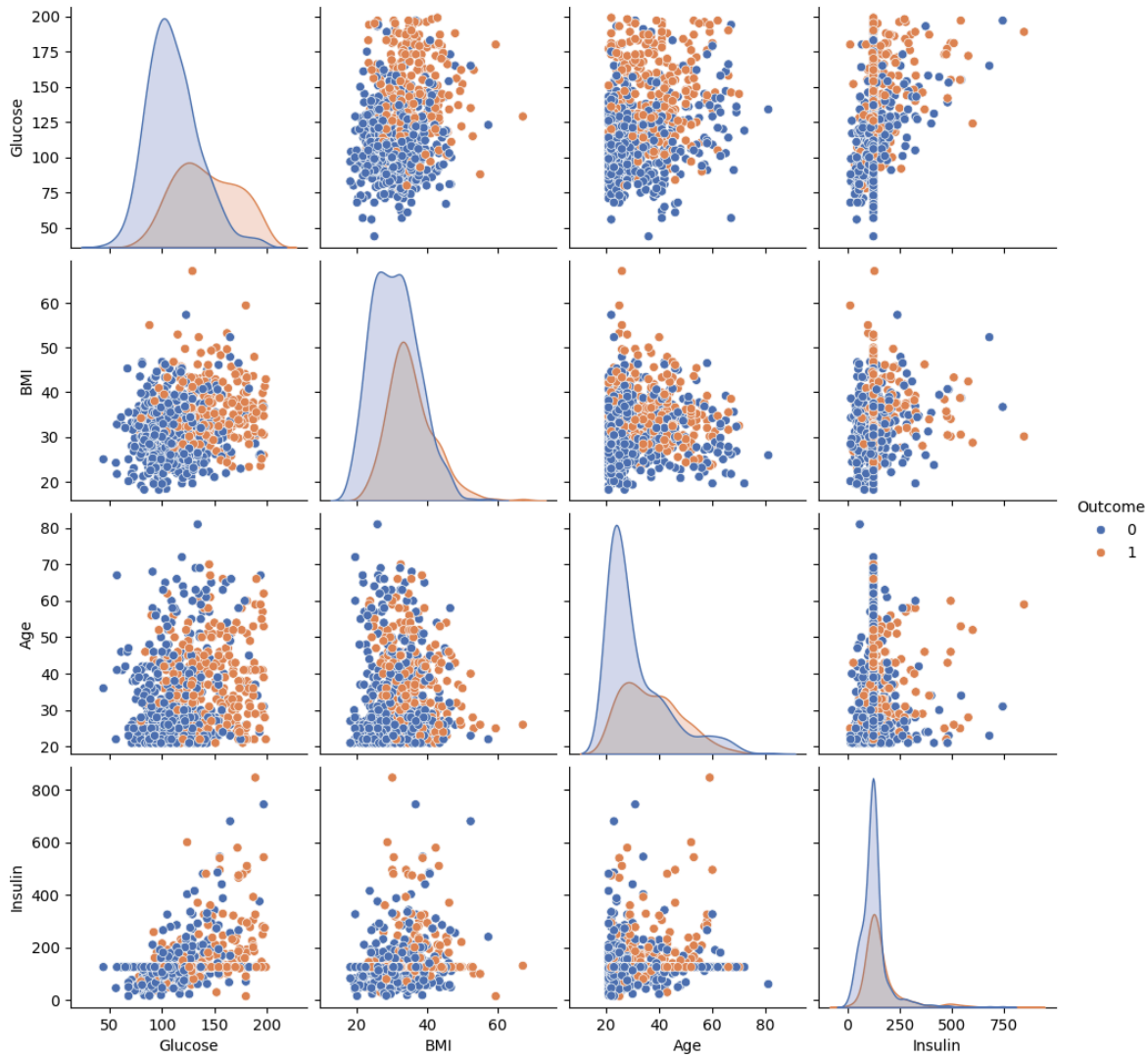



```
features = df_clean.columns.drop('Outcome')
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
```

```
for ax, col in zip(axes.ravel(), features):
    sns.boxplot(x='Outcome', y=col, data=df_clean, ax=ax, palette=['#4C72B0'
    ax.set_title(f'{col} by Outcome')
plt.tight_layout()
plt.show()
```




```
sns.pairplot(df_clean[['Glucose', 'BMI', 'Age', 'Insulin', 'Outcome']], hue=
plt.show()
```


Glucose
BMI
Age
Insulin
Outcome
0
1

5. Train-Test Split and Feature Scaling

```
X = df_clean.drop('Outcome', axis=1)
y = df_clean['Outcome']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
print('Train:', X_train.shape, ' Test:', X_test.shape)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)   # fit on train only (no dat
X_test_scaled = scaler.transform(X_test)

print('Train mean (approx 0):', X_train_scaled.mean(axis=0).round(2))
print('Train std  (approx 1):', X_train_scaled.std(axis=0).round(2))
```

```
Train: (614, 8)  Test: (154, 8)
Train mean (approx 0): [ 0.  0. -0.  0. -0. -0.  0. -0.]
Train std  (approx 1): [1. 1. 1. 1. 1. 1. 1. 1.]
```

6. KNN Model Training and Evaluation (K = 5)

```
K = 5
print('K =', K)
knn = KNeighborsClassifier(n_neighbors=K)
knn.fit(X_train_scaled, y_train)
print('KNN model training successful!')
```

```
K = 5
KNN model training successful!
```

Output 5: Data Standardization and KNN Classification Performance

```
y_pred = knn.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_pred)
print(f'KNN Model Accuracy: {accuracy:.4f}')

y_pred_train = knn.predict(X_train_scaled)
print(f'Training Accuracy : {accuracy_score(y_train, y_pred_train):.4f}')
```

```
KNN Model Accuracy: 0.7208
Training Accuracy : 0.8241
```

```
print('Classification Report:\n')
print(classification_report(y_test, y_pred, target_names=['Non-Diabetic', 'D

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Non-Diabetic', 'Diabetic'], yticklabels=['Non-Diab
```

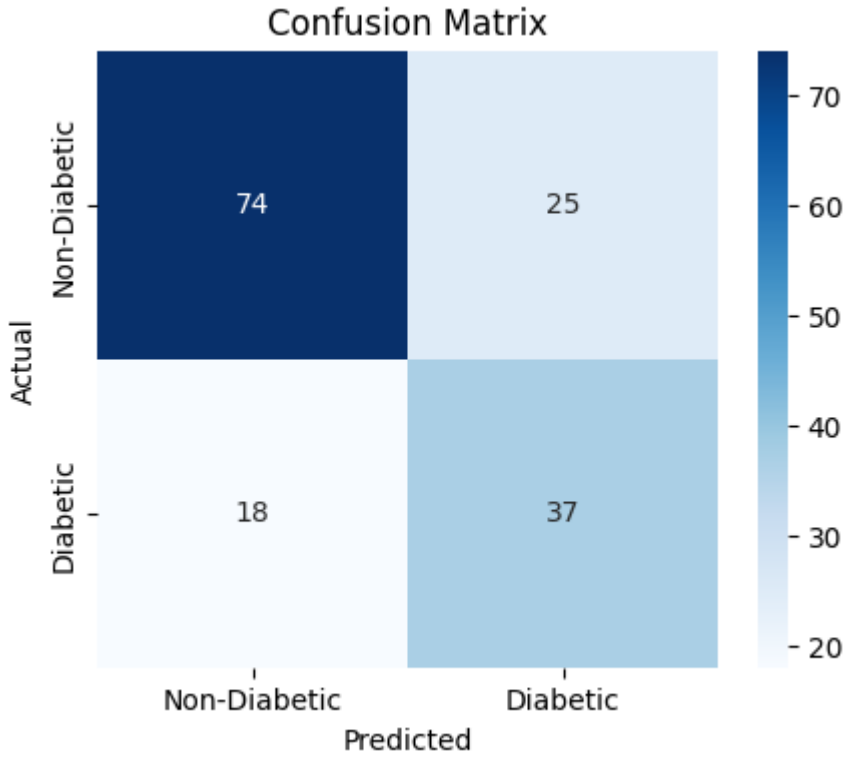
```
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.title('Confusion Matrix')
plt.show()
```

```
Classification Report:

              precision    recall  f1-score   support

Non-Diabetic       0.80      0.75      0.77        99
    Diabetic       0.60      0.67      0.63        55

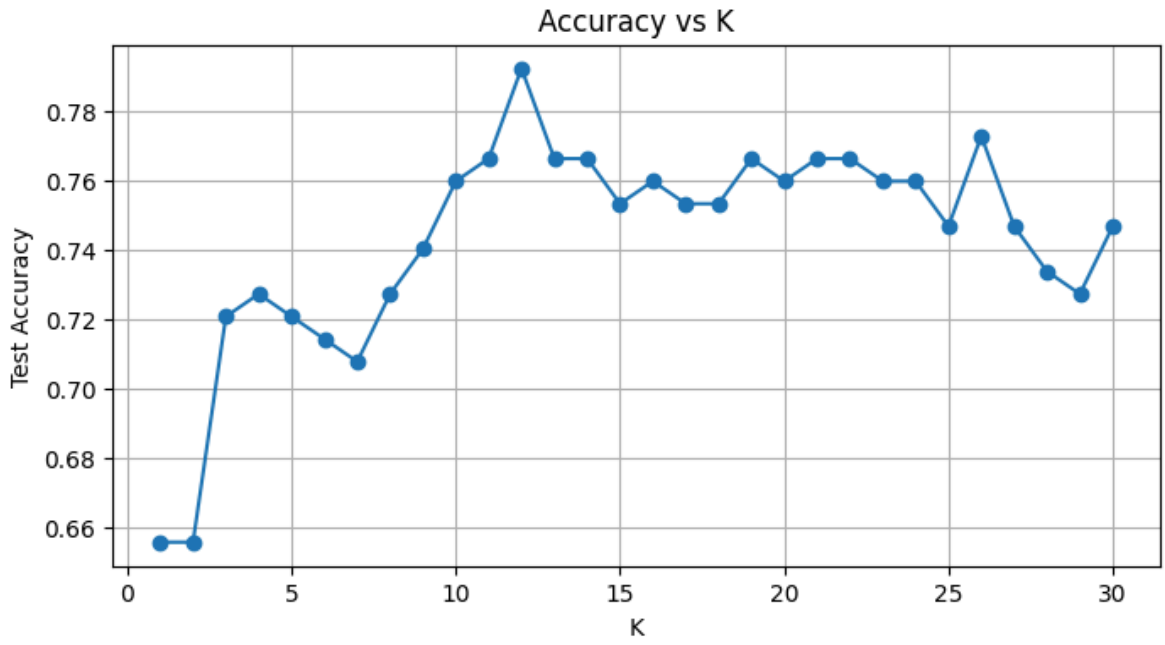
    accuracy                           0.72       154
   macro avg       0.70      0.71      0.70       154
weighted avg       0.73      0.72      0.72       154
```



7. (Optional) Effect of K on Accuracy

```
k_values = range(1, 31)
test_acc = []
for k in k_values:
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train_scaled, y_train)
    test_acc.append(accuracy_score(y_test, m.predict(X_test_scaled)))

plt.figure(figsize=(8, 4))
plt.plot(k_values, test_acc, marker='o')
plt.xlabel('K'); plt.ylabel('Test Accuracy'); plt.title('Accuracy vs K')
plt.grid(True)
plt.show()
print('Best K:', list(k_values)[int(np.argmax(test_acc))], ' Accuracy:', rou
```




```
Best K: 12  Accuracy: 0.7922
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.